

Lab8 文件系统

一、练习1：完成读文件操作的实现

我们先展示完成的代码的内容，然后再对代码进行分析：

代码块

```
1  // (1)处理起始块非对齐的情况
2  blkoff = offset % SFS_BLKSIZE;
3  if (blkoff != 0) {
4      size = (nblks != 0) ? (SFS_BLKSIZE - blkoff) : (endpos - offset);
5      if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
6          return ret;
7      }
8      assert(sfs_block_inuse(sfs, ino));
9      if ((ret = sfs_buf_op(sfs, buf, size, ino, blkoff)) != 0) {
10         return ret;
11     }
12     buf += size;
13     offset += size;
14     alen += size;
15     blkno ++;
16 }
17
18 // (2)处理中间对齐的整块
19 while (nblks > 0 && (endpos - offset) >= SFS_BLKSIZE) {
20     if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
21         return ret;
22     }
23     assert(sfs_block_inuse(sfs, ino));
24     if ((ret = sfs_block_op(sfs, buf, ino, 1)) != 0) {
25         return ret;
26     }
27     buf += SFS_BLKSIZE;
28     offset += SFS_BLKSIZE;
29     alen += SFS_BLKSIZE;
30     blkno ++;
31     nblks --;
32 }
33
34 // (3)处理结束块非对齐的情况
35 if (offset < endpos) {
36     size = endpos - offset;
```

```

37         if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
38             return ret;
39         }
40         assert(sfs_block_inuse(sfs, ino));
41         if ((ret = sfs_buf_op(sfs, buf, size, ino, 0)) != 0) {
42             return ret;
43         }
44         alen += size;
45     }
46

```

sfs_io_nolock()的职责是：根据文件偏移offset，从inode对应的数据块中，把数据拷贝到内存缓冲区buf，它不负责加锁、不负责更新inode元数据，只按块读取文件内容。

然后我们来分析读文件的本质问题：

文件 ≠ 连续磁盘空间，也就是说：

- 文件在逻辑上是连续字节流
- 但在磁盘上是离散的块
- 文件偏移offset可能不在块边界
- 读的长度可能跨多个块

因此，我们需要处理以下三种情况：

情况	说明
起始块	offset不对齐块
中间块	整块对齐
结束块	最后一块不满一个block

因此我们需要在代码中一一解决这些问题。

然后我们对函数中的关键变量进行说明：

代码块

```

1  offset    // 当前文件内偏移
2  endpos    // 本次读操作的结束位置
3  blkno     // 当前逻辑块号 = offset / SFS_BLKSIZE
4  nblks     // 还需要处理的完整块数
5  buf       // 用户缓冲区指针
6  alen      // 实际已读字节数

```

处理起始块“非对齐”情况

代码块

```
1 blkoff = offset % SFS_BLKSIZE;
2 if (blkoff != 0) {
```

触发条件：offset落在某个块的中间。

计算本次最多能读多少字节：

代码块

```
1 size = (nblks != 0) ? (SFS_BLKSIZE - blkoff)
2           : (endpos - offset);
```

含义是：

- 如果后面还有完整块->最多读到当前块结尾
- 如果这是唯一一块->只读到endpos

查inode中该逻辑块对应的物理块

代码块

```
1 ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)
```

这是将逻辑块号->磁盘块号。

名称	含义
blkno	文件内第几个块
ino	实际磁盘块号

从磁盘内的偏移blkoff开始读：

代码块

```
1 ret = sfs_buf_op(sfs, buf, size, ino, blkoff);
```

sfs_buf_op的语义是：从磁盘块ino的blkoff的偏移处，拷贝size字节到buf

更新状态：

代码块

```
1  buf += size;
2  offset += size;
3  alen += size;
4  blkno++;
```

- 缓冲区前移
- 文件偏移前移
- 已读字节数增加
- 进入下一个块

处理中间“整块对齐”情况

代码块

```
1  while (nblks > 0 && (endpos - offset) >= SFS_BLKSIZE) {
```

特点：

- offset正好落在块边界
- 一次可以完整读一个block
- 效率最高的路径

查块映射

代码块

```
1  sfs_bmap_load_nolock(sfs, sin, blkno, &ino)
```

同样是逻辑块->物理块

整块读：

代码块

```
1  sfs_block_op(sfs, buf, ino, 1)
```

`sfs_block_op` 的特点：

函数	用途
sfs_buf_op	读部分块
sfs_block_op	读完整块

传递的参数中的1表示读一个block。

更新状态：

代码块

```
1  buf += SFS_BLKSIZE;
2  offset += SFS_BLKSIZE;
3  alen += SFS_BLKSIZE;
4  blkno++;
5  nblks--;
```

处理结束块“非对齐”情况

代码块

```
1  if (offset < endpos) {
```

触发条件：

- 最后剩余不足一个block
- 必须部分读取

计算剩余字节数：

代码块

```
1  size = endpos - offset;
```

从块头开始读size字节：

代码块

```
1  sfs_buf_op(sfs, buf, size, ino, 0)
```

这里的偏移是0，因为已经对齐到新块起始。

二、练习2：完成基于文件系统的执行程序机制的实现

核心函数实现分析

2.1 load_icode_read函数：文件读取接口

`load_icode_read` 函数是实现文件系统加载的基础工具函数，它封装了文件定位和读取操作。

代码块

```
1  static int
2  load_icode_read(int fd, void *buf, size_t len, off_t offset)
3  {
4      int ret;
5      // 将文件读取位置定位到指定偏移量
6      if ((ret = sysfile_seek(fd, offset, LSEEK_SET)) != 0)
7      {
8          return ret;
9      }
10     // 从文件读取len字节数据到buf中
11     if ((ret = sysfile_read(fd, buf, len)) != len)
12     {
13         // 如果读取的字节数不等于len, 返回错误
14         return (ret < 0) ? ret : -1;
15     }
16     return 0;
17 }
```

功能分析：该函数接受四个参数：文件描述符 `fd`、目标缓冲区 `buf`、读取长度 `len` 和文件偏移量 `offset`。函数首先调用 `sysfile_seek` 将文件指针定位到指定偏移量，然后调用 `sysfile_read` 读取指定长度的数据。这种设计方式将文件操作从原来的内存直接访问改为通过文件系统接口访问，是实现基于文件系统加载的关键。

函数对两个关键步骤都进行了错误检查。如果定位失败或读取的字节数不等于期望值，都会返回错误码。这种严格的错误检查确保了程序加载的可靠性。

2.2 load_icode函数：ELF文件加载核心实现

`load_icode` 函数是实现基于文件系统加载程序的核心函数，负责将ELF格式的可执行文件从文件系统加载到进程内存空间。

2.2.1 参数验证与内存管理初始化

代码块

```
1  static int
```

```

2  load_icode(int fd, int argc, char **kargv)
3  {
4      // 检查参数个数是否合法
5      assert(argc >= 0 && argc <= EXEC_MAX_ARG_NUM);
6      // 确保当前进程的内存管理结构为空 (新进程或exec调用)
7      if (current->mm != NULL) {
8          panic("load_icode: current->mm must be empty.\n");
9      }
10
11     int ret = -E_NO_MEM; // 默认返回值为内存不足错误
12     struct mm_struct *mm;
13
14     // (1) 为当前进程创建新的内存管理结构mm_struct
15     if ((mm = mm_create()) == NULL) {
16         goto bad_mm;
17     }
18
19     // (2) 创建新的页目录表(PDT), mm->pgdir保存页目录的内核虚拟地址
20     if (setup_pgdir(mm) != 0) {
21         goto bad_pgdir_cleanup_mm;
22     }

```

设计思路：与之前实验的主要区别在于，`load_icode` 现在接受的第一个参数是文件描述符 `fd` 而非二进制数据的内存地址。函数首先验证参数个数，确保当前进程的内存管理结构为空，这是为了防止内存泄漏。然后创建新的 `mm_struct` 结构和页目录表，为后续的程序加载做准备。

2.2.2 ELF文件头解析

代码块

```

1      // (3) 将二进制文件的TEXT/DATA/BSS段复制到进程的内存空间
2      struct Page *page;
3      struct elfhdr __elf, *elf = &__elf;
4      struct proghdr __ph, *ph = &__ph;
5      off_t phoff;
6      uint32_t vm_flags, perm;
7
8      // (3.1) 读取文件内容并解析ELF文件头
9      if ((ret = load_icode_read(fd, elf, sizeof(struct elfhdr), 0)) != 0) {
10         goto bad_elf_cleanup_pgdir;
11     }
12
13     // 检查ELF魔数, 验证文件格式是否正确
14     if (elf->e_magic != ELF_MAGIC) {
15         ret = -E_INVALID_ELF;
16         goto bad_elf_cleanup_pgdir;

```

实现要点：这里使用 `load_icode_read` 函数从文件系统读取ELF文件头，而不是直接从内存中读取。ELF文件头包含了程序的元数据，包括魔数、程序入口地址、程序段数量和位置等关键信息。通过验证ELF魔数（0x464C457F），确保加载的是合法的ELF格式文件。

2.2.3 程序段加载：TEXT/DATA段处理

代码块

```
1      // (3.2) 根据ELF文件头中的信息读取并解析程序段头
2      phoff = elf->e_phoff;
3      for (int i = 0; i < elf->e_phnum; i++, phoff += sizeof(struct proghdr)) {
4          // 读取第i个程序段头
5          if ((ret = load_icode_read(fd, ph, sizeof(struct proghdr), phoff)) !=
6              0) {
7              goto bad_cleanup_mmap;
8          }
9          // 只处理LOAD类型的段（需要加载到内存的段）
10         if (ph->p_type != ELF_PT_LOAD) {
11             continue;
12         }
13         if (ph->p_filesz > ph->p_memsz) {
14             ret = -E_INVALID_ELF;
15             goto bad_cleanup_mmap;
16         }
```

段类型过滤：ELF文件包含多种类型的段，但只有 `ELF_PT_LOAD` 类型的段需要加载到内存中。这些段通常包括代码段（TEXT）、数据段（DATA）和BSS段。函数还验证了 `p_filesz <= p_memsz` 的约束，因为文件中的大小不应该超过内存中的大小。

2.2.4 虚拟内存区域映射

代码块

```
1      // (3.3) 调用mm_map为TEXT/DATA段建立虚拟内存区域(VMA)
2      vm_flags = 0, perm = PTE_U | PTE_V;
3      if (ph->p_flags & ELF_PF_X) vm_flags |= VM_EXEC;    // 可执行
4      if (ph->p_flags & ELF_PF_W) vm_flags |= VM_WRITE;   // 可写
5      if (ph->p_flags & ELF_PF_R) vm_flags |= VM_READ;    // 可读
6      // 将VMA标志转换为页表项权限位
7      if (vm_flags & VM_READ) perm |= PTE_R;
8      if (vm_flags & VM_WRITE) perm |= (PTE_W | PTE_R);
9      if (vm_flags & VM_EXEC) perm |= PTE_X;
```



```

10
11      // 在mm中创建虚拟内存区域
12      if ((ret = mm_map(mm, ph->p_va, ph->p_memsz, vm_flags, NULL)) != 0) {
13          goto bad_cleanup_mmap;
14      }

```

权限管理：这段代码展示了从ELF段标志到页表权限的映射过程。ELF格式使用 `ELF_PF_X`（可执行）、`ELF_PF_W`（可写）、`ELF_PF_R`（可读）来描述段的权限。代码首先将这些标志转换为VMA的权限标志，然后进一步转换为RISC-V架构的页表项权限位。特别注意的是，在RISC-V架构中，可写页面必须同时具有可读权限。

2.2.5 从文件加载数据到内存

代码块

```

1      // (3.4) 为TEXT/DATA段分配物理页面，从文件读取内容并复制到新分配的页面
2      off_t offset = ph->p_offset;
3      size_t off, size;
4      uintptr_t start = ph->p_va, end, la = ROUNDDOWN(start, PGSIZE);
5
6      ret = -E_NO_MEM;
7
8      // 处理文件中实际存在的数据部分 (p_filesz字节)
9      end = ph->p_va + ph->p_filesz;
10     while (start < end) {
11         // 为虚拟地址la分配一个物理页
12         if ((page = pgdir_alloc_page(mm->pgdir, la, perm)) == NULL) {
13             goto bad_cleanup_mmap;
14         }
15         // 计算本次复制的起始偏移和大小
16         off = start - la;
17         size = PGSIZE - off;
18         la += PGSIZE;
19         if (end < la) {
20             size -= la - end;
21         }
22         // 从文件读取数据到页面
23         if ((ret = load_icode_read(fd, page2kva(page) + off, size,
offset)) != 0) {
24             goto bad_cleanup_mmap;
25         }
26         start += size, offset += size;
27     }

```

分页加载机制：由于程序段可能跨越多个物理页面，并且起始地址可能不是页对齐的，代码采用了逐页加载的策略。对于每一页，首先通过 `pgdir_alloc_page` 分配物理页面并建立页表映射，然后计算在该页中需要复制的数据范围，最后使用 `load_icode_read` 从文件读取数据到页面中。这里的关键是正确处理页内偏移和跨页边界的情况。

2.2.6 BSS段初始化

代码块

```
1          // (3.5) 为BSS段分配页面并初始化为0
2          end = ph->p_va + ph->p_memsz;
3          if (start < la) {
4              // 处理当前页的剩余部分
5              if (start == end) {
6                  continue;
7              }
8              off = start + PGSIZE - la;
9              size = PGSIZE - off;
10             if (end < la) {
11                 size -= la - end;
12             }
13             memset(page2kva(page) + off, 0, size);
14             start += size;
15             assert((end < la && start == end) || (end >= la && start == la));
16         }
17         // 为剩余的BSS段分配新页面并全部清零
18         while (start < end) {
19             if ((page = pgdir_alloc_page(mm->pgdir, la, perm)) == NULL) {
20                 goto bad_cleanup_mmap;
21             }
22             off = start - la, size = PGSIZE - off, la += PGSIZE;
23             if (end < la) {
24                 size -= la - end;
25             }
26             memset(page2kva(page) + off, 0, size);
27             start += size;
28         }
```

BSS段处理：BSS段是存放未初始化全局变量的区域，在文件中不占用空间但在内存中需要分配并初始化为0。代码通过 `p_memsz - p_filesz` 计算BSS段的大小。如果上一步在页面中间结束，需要先清零当前页的剩余部分，然后为完整的BSS段分配新页面并清零。

2.3 用户栈设置与参数传递

2.3.1 用户栈建立

代码块

```
1 // (4) 建立用户栈的虚拟内存映射
2 vm_flags = VM_READ | VM_WRITE | VM_STACK;
3 if ((ret = mm_map(mm, USTACKTOP - USTACKSIZE, USTACKSIZE, vm_flags, NULL))
    != 0) {
4     goto bad_cleanup_mmap;
5 }
6 // 为用户栈分配物理页面 (至少4页)
7 assert(pgdir_alloc_page(mm->pgdir, USTACKTOP - PGSIZE, PTE_USER) != NULL);
8 assert(pgdir_alloc_page(mm->pgdir, USTACKTOP - 2 * PGSIZE, PTE_USER) !=
    NULL);
9 assert(pgdir_alloc_page(mm->pgdir, USTACKTOP - 3 * PGSIZE, PTE_USER) !=
    NULL);
10 assert(pgdir_alloc_page(mm->pgdir, USTACKTOP - 4 * PGSIZE, PTE_USER) !=
    NULL);
```

栈区域设计：用户栈位于用户空间的高地址区域，从 `USTACKTOP` 向下增长。代码首先通过 `mm_map` 建立栈的虚拟内存区域，然后预先分配4个物理页面。这种预分配策略避免了程序启动时的缺页异常，提高了执行效率。

2.3.2 命令行参数传递

代码块

```
1 // (6) 在用户栈中设置argc和argv参数
2 uintptr_t sp = USTACKTOP;
3 char *uargv_ptrs[EXEC_MAX_ARG_NUM + 1];
4
5 // 第一步：将所有参数字符串复制到用户栈
6 for (int i = 0; i < argc; i++) {
7     int len = strlen(kargv[i]) + 1;
8     sp -= len;
9     uintptr_t dst = sp;
10    char *src = kargv[i];
11    // 将字符串从内核空间复制到用户空间
12    while (len > 0) {
13        struct Page *p = get_page(mm->pgdir, dst, NULL);
14        assert(p != NULL);
15        uintptr_t kva = (uintptr_t)page2kva(p);
16        off_t off = dst % PGSIZE;
17        size_t copylen = PGSIZE - off;
18        if (copylen > len) copylen = len;
19        memcpy((void *) (kva + off), src, copylen);
20        dst += copylen;
21        src += copylen;
```

```

22         len -= copylen;
23     }
24     uargv_ptrs[i] = (char *)sp;
25 }
26 uargv_ptrs[argc] = 0;

```

参数布局设计：命令行参数采用标准的Unix风格布局。首先将所有参数字符串从高地址向低地址依次压入栈中，记录每个字符串在用户空间的地址。由于字符串可能跨越页面边界，代码采用逐页复制的方式，通过 `get_page` 获取目标页面，计算页内偏移，然后进行内存复制。

代码块

```

1 // 第二步：将argv指针数组复制到用户栈
2 sp = ROUNDDOWN(sp, sizeof(uintptr_t));
3 int argv_size = (argc + 1) * sizeof(char *);
4 sp -= argv_size;
5
6 {
7     uintptr_t dst = sp;
8     char *src = (char *)uargv_ptrs;
9     int len = argv_size;
10    while (len > 0) {
11        struct Page *p = get_page(mm->pgdir, dst, NULL);
12        assert(p != NULL);
13        uintptr_t kva = (uintptr_t)page2kva(p);
14        off_t off = dst % PGSIZE;
15        size_t copylen = PGSIZE - off;
16        if (copylen > len) copylen = len;
17        memcpy((void *) (kva + off), src, copylen);
18        dst += copylen;
19        src += copylen;
20        len -= copylen;
21    }
22 }

```

指针数组布局：在字符串下方，按指针大小对齐后，复制argv指针数组。该数组包含指向各个参数字符串的指针，以NULL结尾。这种布局符合C语言main函数的参数传递约定。

2.3.3 进程上下文切换

代码块

```

1 // (5) 设置当前进程的mm和页目录，切换页表
2 mm_count_inc(mm);
3 current->mm = mm;

```

```

4     current->pgdir = PADDR(mm->pgdir);
5     lsatp(PADDR(mm->pgdir));
6
7     // (7) 设置用户环境的trapframe
8     struct trapframe *tf = current->tf;
9     memset(tf, 0, sizeof(struct trapframe));
10    tf->gpr.sp = sp;
11    tf->epc = elf->e_entry;
12    tf->gpr.a0 = argc;
13    tf->gpr.a1 = sp;
14
15    uintptr_t sstatus = read_csr(sstatus);
16    sstatus &= ~SSTATUS_SPP;    // 清除SPP位, 返回用户态
17    sstatus |= SSTATUS_SPIE;    // 设置SPIE位, 返回后启用中断
18    tf->status = sstatus;
19
20    ret = 0;

```

上下文建立：代码完成了从内核态到用户态的完整准备。首先更新进程的内存管理结构和页表，通过 `lsatp` 加载新页表。然后设置trapframe，包括用户栈指针、程序入口地址、参数寄存器和状态寄存器。清除 `SSTATUS_SPP` 位确保返回到用户态，设置 `SSTATUS_SPIE` 位确保返回后中断使能。

2.4 do_execve函数：程序执行入口

代码块

```

1  int do_execve(const char *name, int argc, const char **argv)
2  {
3      static_assert(EXEC_MAX_ARG_LEN >= FS_MAX_FPATH_LEN);
4      struct mm_struct *mm = current->mm;
5
6      if (!(argc >= 1 && argc <= EXEC_MAX_ARG_NUM))
7      {
8          return -E_INVALID;
9      }
10
11     char local_name[PROC_NAME_LEN + 1];
12     memset(local_name, 0, sizeof(local_name));
13     char *kargv[EXEC_MAX_ARG_NUM];
14     const char *path;
15     int ret = -E_INVALID;
16
17     lock_mm(mm);
18
19     // 复制进程名称和参数到内核空间
20     if (name == NULL) {

```

```

21         snprintf(local_name, sizeof(local_name), "<null> %d", current->pid);
22     } else {
23         if (!copy_string(mm, local_name, name, sizeof(local_name))) {
24             unlock_mm(mm);
25             return ret;
26         }
27     }
28
29     if ((ret = copy_kargv(mm, argc, kargv, argv)) != 0) {
30         unlock_mm(mm);
31         return ret;
32     }
33
34     path = argv[0];
35     unlock_mm(mm);

```

参数处理： `do_execve` 是执行新程序的系统调用处理函数。它首先验证参数个数，然后将进程名和所有命令行参数从用户空间复制到内核空间。这是必要的，因为后续会释放当前进程的用户空间，而参数信息需要保留。

代码块

```

1         // 关闭所有打开的文件
2         files_closeall(current->filesp);
3
4         // 打开要执行的可执行文件
5         int fd;
6         if ((ret = fd = sysfile_open(path, O_RDONLY)) < 0) {
7             goto execve_exit;
8         }
9
10        // 回收当前进程的旧内存空间
11        if (mm != NULL) {
12            lsatp(boot_pgdir_pa);
13            if (mm_count_dec(mm) == 0) {
14                exit_mmap(mm);
15                put_pgdir(mm);
16                mm_destroy(mm);
17            }
18            current->mm = NULL;
19        }
20        ret = -E_NO_MEM;
21
22        // 加载新程序
23        if ((ret = load_icode(fd, argc, kargv)) != 0) {
24            goto execve_exit;

```

```
25     }
26
27     put_kargv(argc, kargv);
28     set_proc_name(current, local_name);
29
30     return 0;
```

关键步骤：函数通过 `sysfile_open` 从文件系统打开可执行文件，获得文件描述符。然后切换到内核页表，释放当前进程的所有用户内存。最后调用 `load_icode` 加载新程序。这个过程完成了进程映像的完全替换，是 `exec` 系统调用的核心实现。

与文件系统的集成

3.1 文件操作接口

在本实验中，程序加载不再直接访问内存，而是通过文件系统接口：

- **sysfile_open**：打开可执行文件，返回文件描述符
- **sysfile_seek**：定位文件读写位置
- **sysfile_read**：从文件读取数据
- **files_closeall**：关闭进程的所有打开文件

这些接口将VFS（虚拟文件系统）层与进程管理层连接起来，使得程序可以从磁盘文件系统加载，而不是从静态编译的二进制数据加载。

3.2 copy_files机制

代码块

```
1  static int
2  copy_files(uint32_t clone_flags, struct proc_struct *proc)
3  {
4      struct files_struct *filesp, *old_filesp = current->filesp;
5      assert(old_filesp != NULL);
6
7      if (clone_flags & CLONE_FS) {
8          filesp = old_filesp;
9          goto good_files_struct;
10     }
11
12     int ret = -E_NO_MEM;
13     if ((filesp = files_create()) == NULL) {
14         goto bad_files_struct;
15     }
16
```

```

17     if ((ret = dup_files(filesp, old_files)) != 0) {
18         goto bad_dup_cleanup_fs;
19     }
20
21     good_files_struct:
22         files_count_inc(filesp);
23         proc->files = filesp;
24         return 0;
25
26     bad_dup_cleanup_fs:
27         files_destroy(filesp);
28     bad_files_struct:
29         return ret;
30 }

```

文件描述符继承：在 `do_fork` 中调用 `copy_files`，实现了文件描述符的继承机制。如果设置了 `CLONE_FS` 标志，父子进程共享文件描述符表；否则，复制一份新的文件描述符表。这确保了新创建的进程能够正确地继承或独立管理文件资源。

代码完成后，执行 `make qemu`。发现能看到sh用户程序的执行界面，并且在sh用户界面上可以执行 `exit`、`hello` 等用户程序，说明实验成功。

```

check_vmm() succeeded.
sched class: RR_scheduler
Initrd: 0xc0214010 - 0xc021bd0f, size: 0x00007d00
Initrd: 0xc021bd10 - 0xc029100f, size: 0x00075300
sfs: mount: 'simple file system' (106/11/117)
vfs: mount disk0.
++ setup timer interrupts
kernel_execve: pid = 2, name = "sh".
user sh is running!!!
$

```

三、扩展练习Challenge 1：完成基于“UNIX的PIPE机制”的设计方案

1. 总体设计思路

在 VFS 框架下，引入一种特殊 inode（pipe inode），其数据后端是内存环形缓冲区，并配合锁 + 条件变量解决同步互斥。

代码块

```

1  sys_pipe
2  └─ 创建 pipe 对象
3      └─ pipe inode

```



```
4      |— pipe buffer
5      |— 两个 file (读端 / 写端)
6      |— 挂接 pipe_file_ops
```

2. 核心数据结构

2.1 管道核心结构: `struct pipe`

代码块

```
1  #define PIPE_BUF_SIZE 4096
2  struct pipe {
3      char buffer[PIPE_BUF_SIZE];    // 环形缓冲区
4      size_t rpos;                    // 读指针
5      size_t wpos;                    // 写指针
6      size_t data;                    // 当前已有数据字节数
7
8      int readers;                    // 读端引用计数
9      int writers;                    // 写端引用计数
10
11     struct mutex lock;               // 互斥锁 (保护整个 pipe)
12     struct condvar read_cv;          // 读阻塞队列
13     struct condvar write_cv;         // 写阻塞队列
14 };
```

2.2 Pipe inode 的私有数据

Pipe 不需要磁盘 inode，但仍然要有 VFS inode。

代码块

```
1  struct pipe_inode {
2      struct pipe *pipe;    // 指向真正的 pipe 对象
3  };
```

代码块

```
1  struct inode {
2      ...
3      void *i_private;    // 对 pipe inode 指向 pipe_inode
4      struct inode_ops *i_ops;
5      struct file_ops *f_ops;
6  };
```

3. 提供的接口

3.1 系统调用接口

代码块

```
1  int sys_pipe(int fd[2]);
```

- 创建一个新的管道
- 返回两个文件描述符：
 - `fd[0]`：读端
 - `fd[1]`：写端
- 两个 fd 指向同一个 pipe

3.2 VFS 层接口 (file_ops)

代码块

```
1  struct file_ops pipe_file_ops = {  
2      .read  = pipe_read,  
3      .write = pipe_write,  
4      .close = pipe_close,  
5  };
```

```
ssize_t pipe_read(struct file *file, void *buf, size_t len);
```

- 若 `data > 0`：
 - 从 buffer 读，更新 `rpos/data`
 - 唤醒等待写的进程
- 若 `data == 0`：
 - 若 `writers == 0`：返回 0 (EOF)
 - 否则：阻塞在 `read_cv`

```
ssize_t pipe_write(struct file *file, const void *buf, size_t len);
```

- 若 `readers == 0`：
 - 返回错误（或向进程发送 SIGPIPE）
- 若 buffer 未滿：

- 写入数据，更新 `wpos/data`
- 唤醒等待读的进程
- 若 buffer 已满：
 - 阻塞在 `write_cv`

```
int pipe_close(struct file *file);
```

- 若关闭的是读端：
 - `readers--`
 - 若 `readers == 0`：唤醒写端
- 若关闭的是写端：
 - `writers--`
 - 若 `writers == 0`：唤醒读端
- 若 `readers == 0 && writers == 0`：
 - 释放 pipe 对象

4. 同步与互斥设计说明

所有对 pipe 状态（buffer / pos / data / refcnt）的访问，必须在 `pipe->lock` 保护下进行。

操作	阻塞条件	唤醒者
read	<code>data == 0</code> 且 <code>writers > 0</code>	write / close
write	<code>data == PIPE_BUF_SIZE</code>	read
read EOF	<code>data == 0</code> 且 <code>writers == 0</code>	—
write error	<code>readers == 0</code>	—

四、扩展练习Challenge 2：完成基于“UNIX的软连接和硬连接机制”的设计方案

1. 总体设计思路

硬链接

- 本质：增加一个目录项，指向已有 inode
- 不引入新 inode

- 依赖 inode 的 `nlink` 计数

软链接

- 本质：一个特殊类型的 inode
- inode 的数据区存储“目标路径字符串”
- 路径解析阶段进行展开

全部接入 VFS

- 操作通过 VFS 接口完成
- 具体文件系统（如 SFS）实现底层细节

并发安全

- inode / dentry / 目录内容的修改必须加锁
- 防止竞态条件（link/unlink/open 并发）

2. 核心数据结构

2.1 inode 结构的扩展

代码块

```
1  struct inode {
2      int i_type;           // I_REG, I_DIR, I_SYMLINK, ...
3      int i_nlink;          // 硬链接计数
4      size_t i_size;
5
6      struct inode_ops *i_ops;
7      struct file_ops *f_ops;
8
9      void *i_private;       // 文件系统私有数据
10
11     struct mutex i_lock;    // 保护 inode 元数据
12 };
```

2.2 目录项（dentry）结构

代码块

```
1  struct dentry {
2      char name[MAX_NAME_LEN];
3      struct inode *inode;
4
5      struct dentry *parent;
```

```
6     struct mutex d_lock;    // 保护该目录项
7 };
```

2.3 软链接 inode 的私有数据

代码块

```
1     struct symlink_info {
2         char *target_path;    // 目标路径字符串
3     };
```

在 inode 中：

代码块

```
1     inode->i_type = I_SYMLINK;
2     inode->i_private = symlink_info;
```

3. 提供的接口

3.1 系统调用接口

硬链接

```
1     int sys_link(const char *oldpath, const char *newpath);
```

- 解析 `oldpath`，得到目标 inode
- 在 `newpath` 所在目录创建新 dentry
- 新 dentry 指向已有 inode
- inode 的 `i_nlink++`
- 不允许：
 - 目标为目录（简化设计）
 - 跨文件系统

删除链接（通用）

```
1     int sys_unlink(const char *path);
```

- 删除路径对应的目录项
- inode 的 `i_nlink--`
- 若：
 - `i_nlink == 0`
 - 且没有进程打开
- 则释放 inode 及其数据

软链接

```
1  int sys_symlink(const char *target, const char *linkpath);
```

语义：

- 创建一个新 inode，类型为 `I_SYMLINK`
- inode 数据存储 `target` 字符串
- 在 `linkpath` 处创建目录项
- 不要求 target 存在

读取软链接内容

```
1  ssize_t sys_readlink(const char *path, char *buf, size_t size);
```

- 若 inode 类型不是 symlink → 返回错误
- 将目标路径字符串拷贝到用户缓冲区
- 不展开链接

3.2 VFS 层 inode 操作接口

代码块

```
1  struct inode_ops {
2      int (*lookup)(struct inode *dir, const char *name,
3                    struct inode **result);
4
5      int (*link)(struct inode *dir, struct inode *inode,
6                  const char *name);
7
8      int (*unlink)(struct inode *dir, const char *name);
9
10     int (*readlink)(struct inode *inode, char *buf, size_t size);
```

```
11     };
```

- `link`
 - 在目录 `dir` 中添加 `<name, inode>`
- `unlink`
 - 删除目录项
- `readlink`
 - 仅对 symlink inode 有效

3.3 路径解析阶段对软链接的支持

VFS 层逻辑（伪代码）

```
1  lookup(path):
2      if inode.type == I_SYMLINK:
3          read target_path
4          restart path resolution with target_path
```

设置最大展开深度，防止循环。

4. 同步与互斥设计说明

4.1 inode 锁

所有对 `i_nlink` / `i_type` / `i_private` 的修改，必须持有 `inode->i_lock`。

4.2 目录项锁

所有对目录内容的增删（link/unlink），必须持有父目录 inode 锁 + dentry 锁。

4.3 释放条件

inode 只有在 `i_nlink == 0 && open_file_count == 0` 时才能释放。

五、实验中的重要知识点

1. UNIX 文件系统的三层核心抽象（非常重要）

1.1 文件（File）：本质是“字节序列”

在 UNIX 中：

- 文件是**无结构的字节流**
- 内核不关心内容格式（文本 / ELF / 图片都一样）
- 一切皆文件，包括普通文件、目录、设备文件、管道、socket等等

1.2 inode (index node)：UNIX 文件系统的核心抽象

每个文件 **唯一对应一个 inode**，inode 中保存文件类型、权限、所有者、文件大小、时间戳、数据块指针等内容。

inode 不保存文件名。

1.3 目录 (Directory)：名字 → inode 的映射表

目录在 UNIX 中：

- **本质也是文件**
- 内容是若干条记录： `<filename, inode_number>`

所以对于 `/home/user/a.txt`，

查找过程是：

代码块

```
1  "/"  → inode(2)
2  home → inode(x)
3  user → inode(y)
4  a.txt → inode(z)
```

2. 通用文件模型 (Common File Model, CFM)

通用文件模型是一种内核级抽象：用统一的数据结构和操作接口，描述和访问所有“文件类对象”，屏蔽底层文件系统或设备的差异。也就是上层（系统调用、用户程序）只看到“文件”，下层（ext4 / FAT / pipe / device）各自实现细节。

CFM 中的“四大核心抽象”

1. inode (索引节点) —— “文件是什么”

- 描述一个**文件对象本身**
- 独立于进程
- 不关心“谁在用它”

inode存储关于某个文件的元数据信息（如访问控制权限、大小、拥有者、创建时间、数据内容等等），通常对应磁盘上的**文件控制块**（file control block）。每个索引节点有一个编号，唯一确定文件系统里的一个文件。

2. file（文件）——“一次打开的文件”

- 每次 `open()` 都会创建一个
- **进程相关**
- 保存运行时状态

这里定义的file object不是指磁盘上的一个“文件”，而是指一个进程和它打开的一个文件之间的关系，这个对象存储在内核态的内存中，仅当某个进程打开某个文件的时候才存在。

3. dentry（目录项）——“名字 ↔ inode”

- 解决“文件叫什么”
- 用于路径解析

一个目录也是一个文件，包含若干个子目录和其他文件。从某个子目录、文件的名称，对应到具体的文件/子目录的地址(或者索引节点 `inode`) 的连接，通过**目录项**(dentry)来描述。

本质是：维护从“目录里的某一项”到“对应的文件”的连接/指针

```
"hello.txt" → inode*
```

4. superblock（超级块）——“文件系统整体”

- 描述一个已挂载的文件系统
- 每个挂载点一个 superblock

包含：

- block size
- inode 管理方式
- 根 inode
- fs_ops

存储整个文件系统的相关信息。对于磁盘上的文件系统，对应磁盘里的**文件系统控制块**(filesystem control block)。

六、其他重要知识点

一次磁盘访问时间主要包含：

- **寻道时间（Seek Time）** ← 最主要优化目标
- 旋转延迟（Rotation Delay）
- 传输时间（Transfer Time）

常见的磁盘调度算法

1. FCFS (First-Come First-Served, 先来先服务)

按请求到达顺序处理。实现简单，但性能差，磁头可能疯狂来回移动。

2. SSTF (Shortest Seek Time First, 最短寻道优先)

优先处理“离当前磁头最近”的请求。平均寻道时间短，但可能产生饥饿。

3. SCAN (电梯算法)

磁头像电梯一样朝一个方向扫描，处理沿途所有请求，到头后再反向。饥饿问题大大缓解，但边缘请求等待时间可能较长。

4. C-SCAN (Circular SCAN, 循环扫描)

只在一个方向服务请求，到达末端后直接“跳回起点”。等待时间更均匀，但有一次“空跳”。

5. C-LOOK

磁臂先到达该方向上最后一个请求处，然后立即反转，而不是先到最后点路径上的所有请求。能减少无意义的磁头移动，实际系统中更常见。